

**Propuesta de Arquitectura Omnicanal y
Ecosistema de Microservicios
Banca Virtual BP**

Contenido

1. Introducción	3
2. Alcance del Proyecto	3
3. Cumplimiento Normativo y de Seguridad	4
3.1. Privacidad y Seguridad Transaccional	4
3.2. Cifrado y Seguridad Perimetral	4
4. Definición del Stack Tecnológico	4
4.1. Decisiones Estructurales Críticas	4
4.2. Arquitectura de Canales	6
Para ofrecer una experiencia de usuario fluida, segura y de baja latencia, se ha diseñado una arquitectura de canales digitales evaluando las principales alternativas del mercado para cumplir con los requerimientos transaccionales y de desarrollo multiplataforma.	6
4.3. Computación y Diseño de Microservicios	6
4.4. Arquitectura de Datos	7
4.5. Capa de Integración y Caché	7
4.6. Event-Driven Architecture y Mensajería	8
4.7. Servicios de Identidad	9
4.8. Notificaciones Multicanal	9
4.9. Observabilidad, Logs de Errores y Almacenamiento a Largo Plazo	10
4.10. Patrones de Resiliencia Síncrona: Circuit Breaker y Retry	10
5. Diseño Arquitectónico	10
5.1. Nivel 1: Diagrama de Contexto	10
5.2. Nivel 2: Diagrama de Contenedores	11
5.3. Nivel 3: Diagrama de Componentes y Patrones de Diseño	13
6. Infraestructura Cloud, Alta Disponibilidad y Recuperación ante Desastres	14
6.1. Alta Disponibilidad (HA) y Tolerancia a Fallos	14
6.2. Auto-Healing y Excelencia Operativa	15
6.3. Recuperación ante Desastres y Backups	15
6.4. Estrategia de Manejo de Costos	15
7. Conclusiones y Valor Agregado de la Solución	15

1. Introducción

El presente documento detalla la propuesta de arquitectura técnica para el sistema de Banca en Línea de la entidad financiera BP. Este diseño responde a la necesidad de evolución digital de la institución, proponiendo una solución escalable, segura y altamente disponible.

La arquitectura adopta un enfoque basado en Microservicios alineado a los principios de Domain-Driven Design, lo que permite delimitar claramente los dominios de negocio y técnicos. Para garantizar la máxima resiliencia, el ecosistema se diseña bajo una Arquitectura Orientada a Eventos y el patrón, separando las cargas de lectura y escritura.

Toda la infraestructura se desplegará en la nube de AWS bajo un modelo serverless y de excelencia operativa, garantizando el cumplimiento estricto de la normativa SEPS y la Ley de Protección de Datos Personales.

2. Alcance del Proyecto

El sistema abarca el diseño bajo el modelo C4 de canales digitales (SPA y Móvil) soportados por un ecosistema de microservicios especializados. El alcance se divide en los siguientes dominios:

Dominios Core de Negocio:

- **Consulta de Productos, Movimientos y Datos Básicos:** Servicio de lectura de alta disponibilidad encargado de la composición y agregación de datos. Orquesta peticiones en tiempo real hacia el Core Financiero y el Sistema Independiente, apoyándose en estrategias de caché distribuida para garantizar tiempos de respuesta óptimos y proteger a los sistemas legados de sobrecargas.
- **Transferencias:** Bounded Context transaccional de misión crítica responsable de las transferencias, tanto internas como interbancarias. Implementa patrones de alta resiliencia para sus integraciones síncronas mediante Circuit Breaker y garantiza la consistencia eventual del sistema mediante la emisión segura de eventos de dominio utilizando el patrón Transactional Outbox.

Dominios de Soporte y Transversales:

- **Auditoría, Logs y Vista de Datos:** Microservicio centralizado encargado de registrar de forma inmutable todas las acciones de los usuarios. Adicionalmente, actúa como el motor de datos que procesa los eventos del bus para identificar y persistir la información de clientes frecuentes en una base de datos de lectura optimizada, garantizando la segregación de responsabilidades.
- **Identidad y Onboarding:** Flujo de enrolamiento con reconocimiento facial e integración con el Registro Civil para la validación de identidad, sumado a la delegación de la autenticación y autorización segura mediante estándares OAuth 2.0.

- **Notificaciones:** Sistema multicanal vía Email y SMS desacoplado mediante buses de mensajería para el envío de alertas transaccionales y códigos de seguridad, asegurando que el sistema sea resiliente ante fallos en los proveedores externos.

3. Cumplimiento Normativo y de Seguridad

La arquitectura garantiza la operación segura bajo la Normativa SEPS 009 y la LOPDP:

3.1. Privacidad y Seguridad Transaccional

Autenticación Delegada: Se implementa el flujo OAuth2.0 y Authorization Code Flow with PKCE. Al tener clientes públicos que no pueden almacenar un Client Secret de forma segura, PKCE garantiza que un atacante no pueda interceptar ni inyectar códigos de autorización, siendo el estándar actual para canales digitales.

Consentimiento Biométrico y LOPDP: La aplicación gestiona el consentimiento explícito para el tratamiento de datos biométricos sensibles durante el onboarding.

Auditoría Inmutable: Registro persistente de direcciones IP, Device ID y geolocalización.

Prevención de Fraude y Accesos: MFA obligatorio para transferencias, bloqueo por intentos fallidos (3 máximo), y cierre de sesión por inactividad.

3.2. Cifrado y Seguridad Perimetral

Protección de Datos: Anonimización/Cifrado a nivel de campo para PII en las bases de datos.

Seguridad de Red: Cifrado en tránsito, y protección en el borde con AWS WAF para mitigar ataques DDoS y Rate Limiting.

4. Definición del Stack Tecnológico

Cada decisión arquitectónica responde a patrones empresariales y garantiza eficiencia operativa:

4.1. Decisiones Estructurales Críticas

Uso de AWS como Cloud Principal:

Alternativas evaluadas: Microsoft Azure y Google Cloud Platform.

Justificación 1 (Ecosistema Serverless Maduro): AWS ofrece la integración nativa más sólida del mercado para arquitecturas orientadas a eventos mediante servicios serverless (Fargate, DynamoDB, SNS/SQS). Esto reduce radicalmente la carga operativa frente a soluciones equivalentes en otras nubes.

Justificación 2 (Cumplimiento y Latencia): AWS cuenta con un marco de cumplimiento estricto para el sector financiero y dispone de infraestructura de red que garantiza menor latencia en el consumo de APIs.

Elección de Arquitectura Orientada a Eventos:

Alternativas evaluadas: Arquitectura Síncrona tradicional.

Justificación 1 (Desacoplamiento Estricto): EDA permite que el dominio de Transferencias publique un hecho y continúe su ejecución, sin depender del tiempo de respuesta o disponibilidad del servicio de Auditoría o Notificaciones.

Justificación 2 (Resiliencia ante Picos): El uso de colas de mensajes absorbe picos transaccionales abruptos. Si la base de datos de auditoría se ralentiza, los mensajes se encolan sin causar timeouts en la experiencia del cliente.

Uso de Domain-Driven Design:

Alternativas evaluadas: Arquitectura Monolítica o Modular basada puramente en capas técnicas.

Justificación 1 (Alineación con el Negocio): La complejidad de un banco exige modelar el software exactamente como opera la institución. DDD aísla la lógica pura del negocio de la infraestructura técnica, facilitando la auditoría del código.

Justificación 2 (Independencia de Evolución): Al dividir el sistema en Bounded Contexts, equipos distintos pueden actualizar módulos independientes sin riesgo de provocar daños a la plataforma.

Uso de Microservicios:

Alternativas evaluadas: Arquitectura Monolítica.

Justificación 1 (Escalabilidad Asimétrica): Los microservicios permiten asignar más recursos computacionales de forma independiente, optimizando los costos.

Justificación 2 (Aislamiento de Fallos - Bulkhead): Si un microservicio sufre una caída por un error de código, los usuarios aún podrán acceder al resto del sistema. El sistema se degrada de forma elegante en lugar de colapsar por completo.

4.2. Arquitectura de Canales

Para ofrecer una experiencia de usuario fluida, segura y de baja latencia, se ha diseñado una arquitectura de canales digitales evaluando las principales alternativas del mercado para cumplir con los requerimientos transaccionales y de desarrollo multiplataforma.

Aplicación Web: Se evaluaron dos de los principales frameworks/librerías del mercado para el desarrollo del portal web transaccional bajo el patrón SPA:

- **Opción 1:** Angular. Es un framework robusto y altamente estructurado, tradicionalmente usado en banca corporativa. Sin embargo, su curva de aprendizaje es pronunciada y el peso del bundle inicial puede afectar negativamente los tiempos de carga en conexiones móviles lentas o inestables.
- **Opción 2: React (Seleccionada).** Se elige esta librería para la construcción de una SPA pura y altamente dinámica. Su arquitectura basada en un Virtual DOM permite actualizaciones de interfaz ultrarrápidas y una navegación fluida sin recargas de página una vez que el usuario está autenticado. Para su despliegue, la aplicación pre-compilada se distribuye a través de Amazon CloudFront, garantizando la entrega global de los recursos estáticos con latencia mínima y seguridad perimetral integrada. Adicionalmente, facilita la conversión de la plataforma en una Progressive Web App instalable en escritorio.

Aplicación Móvil Multiplataforma: Para la construcción de los canales móviles, se requiere el uso de una solución multiplataforma que permita mantener un código base unificado para los sistemas operativos iOS y Android, asegurando un rendimiento óptimo. Se evaluaron las siguientes alternativas:

- **Opción 1:** Flutter. Ofrece un rendimiento visual excelente y un motor de renderizado propio que garantiza interfaces idénticas en ambas plataformas. Su principal desventaja arquitectónica en el contexto de esta propuesta es su desconexión con el stack tecnológico del backend, lo que imposibilita la reutilización de código o librerías entre el servidor y la aplicación cliente.
- **Opción 2: .NET MAUI (Seleccionada).** Se elige por su profunda madurez en entornos corporativos financieros y su capacidad de generar binarios verdaderamente nativos. La gran ventaja estratégica de esta opción es su integración nativa con el ecosistema .NET. Esto permite compartir librerías de lógica de negocio y validación de dominio directamente entre los microservicios y la aplicación móvil. Al unificar el lenguaje de programación en todo el proyecto, se elimina la duplicidad de reglas de código y se garantiza una implementación más robusta de estándares de seguridad.

4.3. Computación y Diseño de Microservicios

AWS Fargate:

Aislamiento por Dominio: Cada Bounded Context se encapsula en su propio contenedor independiente, evitando que la falla de un dominio afecte a otro.

Estrategia FinOps y Auto-Scaling: Elimina la gestión de servidores y escala automáticamente ante ráfagas de usuarios.

4.4. Arquitectura de Datos

Amazon Aurora (Capa de Persistencia Relacional)

- **Alternativas evaluadas:** Amazon RDS tradicional, Motores NoSQL.
- **Justificación 1 (Garantías Transaccionales ACID y Patrón Outbox):** El uso de un motor relacional es mandatorio para los procesos críticos del negocio. Aurora garantiza atomicidad, consistencia, aislamiento y durabilidad absolutas. Esto es vital para las transferencias y para la implementación del patrón Transactional Outbox, asegurando que el registro financiero y la intención de emitir el evento asíncrono se persistan en una única transacción indivisible, evitando cualquier pérdida o corrupción de datos.
- **Justificación 2 (Soporte CQRS y Escalabilidad de Lecturas):** Gracias a su almacenamiento distribuido y soporte nativo para réplicas de lectura de latencia mínima, Aurora soporta eficientemente el modelo de consultas del patrón CQRS. Dominios de alta demanda pueden acceder a la información en milisegundos sin afectar ni competir por recursos de cómputo con la instancia principal dedicada exclusivamente a las transacciones de escritura del Core.

Amazon DynamoDB (Capa NoSQL - Auditoría y Eventos)

- **Alternativas evaluadas:** Amazon Aurora, MongoDB.
- **Justificación 1 (Alta Ingesta y Escalabilidad Horizontal):** Se descartó una base relacional para este dominio debido a que la auditoría exige persistir un volumen masivo de eventos sin necesidad de esquemas rígidos ni uniones complejas. DynamoDB permite absorber esta carga de escritura masiva con latencia de un solo dígito de milisegundo, escalando de forma elástica según la demanda transaccional del banco.
- **Justificación 2 (Operación Serverless e Inmutabilidad):** Se prefirió frente a bases de datos documentales como MongoDB por ser un servicio completamente administrado, lo que elimina la carga operativa de gestionar, parchear y escalar clústeres. Además, para el caso de uso específico de Auditoría, su integración nativa con AWS IAM permite implementar políticas de acceso estrictas a nivel de tabla, garantizando que los registros sean de solo escritura y criptográficamente inmutables, un requerimiento regulatorio fundamental en la banca para prevenir la manipulación de rastros transaccionales.

4.5. Capa de Integración y Caché

Amazon API Gateway:

Alternativas evaluadas: Acceso directo desde el frontend a los balanceadores de carga de cada microservicio.

Justificación 1 (Centralización Perimetral de Seguridad): Actúa como un escudo único para aplicar políticas globales: validación de firmas JWT, limitación de cuotas y mitigación de ataques, antes de que el tráfico malicioso toque la lógica de negocio.

Justificación 2 (Desacoplamiento Cliente-Servidor): Oculta la topología interna. El frontend móvil consume una única API unificada, permitiendo que el backend se fragmente o consolide en el futuro sin requerir actualizaciones en las tiendas de aplicaciones.

Amazon ElastiCache:

Justificación 1 (Rendimiento de Nueva Generación y Escalabilidad): Se selecciona Valkey como motor de caché por ser una solución optimizada para cargas de trabajo masivas en la nube, ofreciendo tiempos de respuesta de submilisegundos y una gestión eficiente de hilos. Su arquitectura permite garantizar alta disponibilidad en múltiples zonas con conmutación automática, asegurando que la capa de datos temporales sea resiliente y capaz de escalar horizontalmente según la demanda de los canales digitales.

Justificación 2 (Protección y Resiliencia del Core Bancario): Actúa como un componente crítico de mitigación de carga. Al almacenar en memoria los datos de acceso frecuente, absorbe los picos de tráfico transaccional. Esto reduce drásticamente la latencia percibida por el cliente y evita saturar de consultas redundantes al Core Financiero, protegiendo este sistema de posibles degradaciones de servicio por sobrecarga.

4.6. Event-Driven Architecture y Mensajería

Amazon SQS, SNS y Transactional Outbox:

Alternativas evaluadas: Apache Kafka, RabbitMQ.

Justificación 1 (Simplicidad y Costo): Se descartó Kafka por su alta complejidad operativa y costos base elevados. SQS/SNS cubren sobradamente las necesidades de publicación/suscripción con un modelo de precios 100% pago por uso.

Justificación 2 (Cero Administración): Al ser completamente serverless, elimina la necesidad de parchear sistemas operativos o gestionar clústeres subyacentes, alineándose al principio de Excelencia Operativa.

Garantía de Entrega: El evento se guarda en la misma transacción de Aurora que la transferencia. Un worker lee la bandeja de salida y publica en SNS, garantizando que jamás se pierda una alerta por caídas de red.

4.7. Servicios de Identidad

Amazon Rekognition y Registro Civil:

Alternativas evaluadas: Azure Face API, CompreFace.

Justificación 1 (Privacidad y LOPDP): Al estar integrado dentro del mismo ecosistema de AWS, las imágenes biométricas sensibles no transitan por internet público hacia un proveedor de terceros, facilitando el cumplimiento de la LOPDP y la seguridad de red.

Justificación 2 (Costos): Elimina las costosas tarifas por validación de identidad que cobran otras plataformas, pagando fracciones de centavo por el uso directo del API de Liveness y Face Match.

Validación KYC: El Registro Civil actúa como fuente de verdad oficial para la foto del cliente.

Autenticación Subsecuente:

Una vez completado el proceso de onboarding, el acceso recurrente al sistema se registrará por un esquema de doble factor de autenticación para mitigar riesgos de suplantación. El usuario deberá ingresar sus credenciales únicas. El sistema exigirá la validación de un código OTP, enviado de forma asíncrona a través de los canales de notificación implementados.

4.8. Notificaciones Multicanal

Enfoque Híbrido:

Eclipsoft (SMS):

Alternativas evaluadas: Twilio.

Justificación 1 (Ruteo Local Directo): A diferencia de Twilio, Eclipsoft cuenta con conectividad directa con las operadoras ecuatorianas. Esto elimina los filtros anti-spam de SMS internacionales y garantiza la entrega del código OTP en menos de 3 segundos y de las diferentes notificaciones.

Justificación 2 (Facturación y Soporte local): Permite mantener contratos con regulaciones locales, además de soporte técnico cercano.

Amazon SES (Email):

Servicio gestionado con alta reputación de IP para asegurar la entrega masiva de notificaciones sin caer en Spam.

4.9. Observabilidad, Logs de Errores y Almacenamiento a Largo Plazo

Sentry:

Alternativas evaluadas: Amazon CloudWatch Logs, Datadog.

Justificación 1 (Error Tracking Semántico): Sentry se especializa en la capa de código. Agrupa excepciones idénticas de forma inteligente y expone la traza de la pila exacta para el desarrollador.

Justificación 2 (Ecosistema Frontend/Mobile): Se prefirió por su excelente integración nativa con React y .NET MAUI, capturando no solo el error del servidor, sino el estado del dispositivo móvil en el momento de la falla.

4.10. Patrones de Resiliencia Síncrona: Circuit Breaker y Retry

Para gestionar la comunicación síncrona con servicios externos, se implementan patrones de tolerancia a fallos a nivel de código en los microservicios.

Justificación 1 (Prevención de Fallos en Cascada - Circuit Breaker): Si un servicio externo presenta latencia extrema o caídas, el Circuit Breaker se abre automáticamente, rechazando nuevas peticiones de inmediato. Esto evita que los contenedores de Fargate agoten sus hilos de ejecución y colapsen esperando respuestas que no llegarán.

Justificación 2 (Retry con Exponential Backoff): Ante fallos de red momentáneos, el sistema reintenta la petición de forma automática incrementando el tiempo de espera entre cada intento.

5. Diseño Arquitectónico

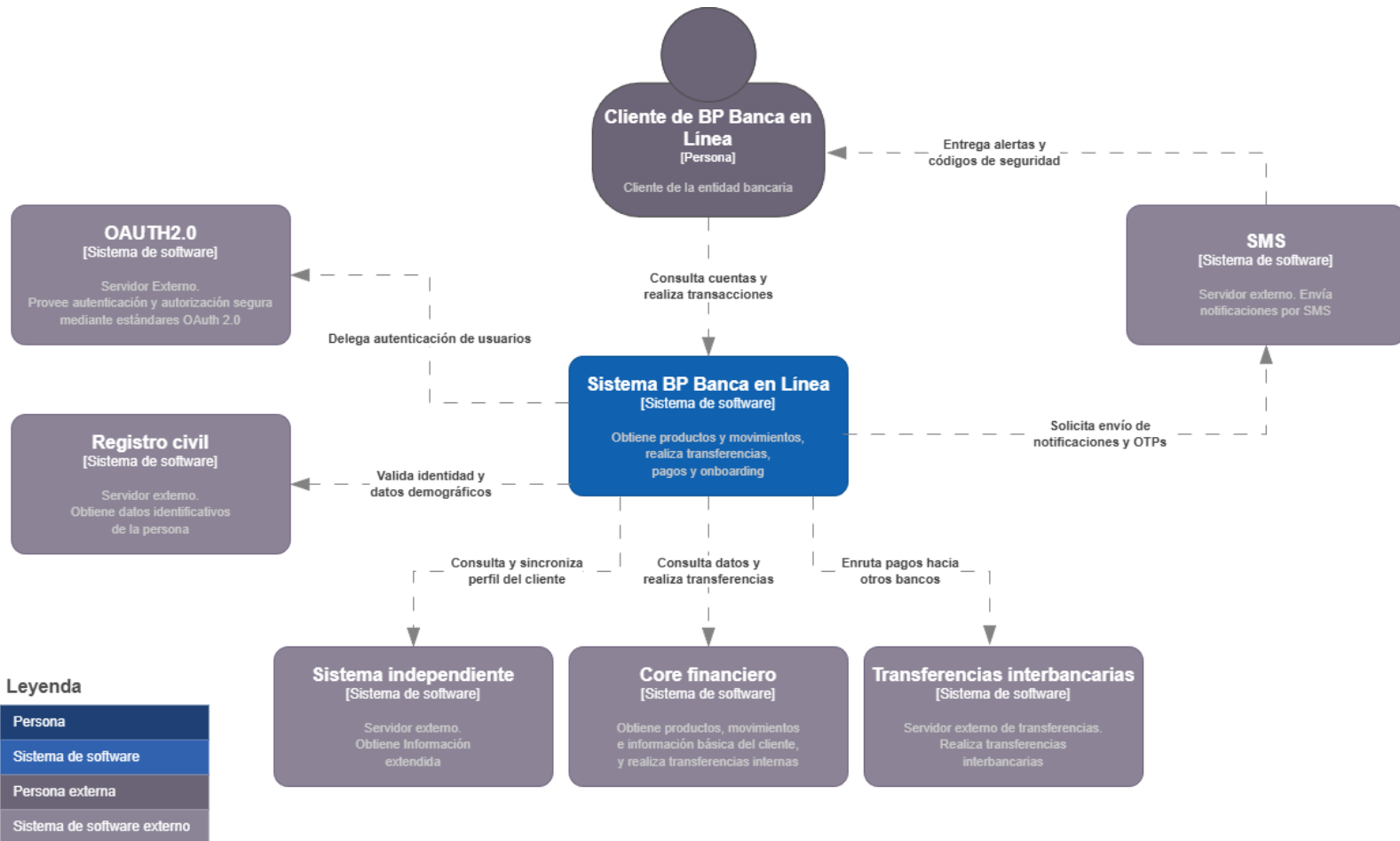
La arquitectura de la plataforma de Banca en Línea se ha diseñado utilizando el estándar de modelado C4, permitiendo una comprensión progresiva del sistema desde su contexto global hasta sus componentes internos. El diseño se rige por los principios de Domain-Driven Design para el aislamiento de contextos de negocio y opera bajo una Arquitectura Orientada a Eventos para garantizar la resiliencia.

5.1. Nivel 1: Diagrama de Contexto

Descripción de la Interacción Global: Este nivel ilustra la plataforma como un orquestador central que interactúa con su entorno bancario. Los actores principales consumen la Plataforma BP Banca en Línea para gestionar sus finanzas de forma omnicanal. El sistema se integra con los siguientes sistemas externos críticos:

- **Fuentes de Datos y Transferencias:** Conexión con el Core Financiero para transferencias de saldos y la Red Interbancaria para el enrutamiento de transferencias externas.

- **Gestión de Identidad y Sistema Independiente:** Validación de datos demográficos ante el Registro Civil del Ecuador y sincronización de perfiles extendidos con el Sistema Independiente.
- **Proveedores de Comunicación y Seguridad:** Interacción con Eclipsoft para envío de SMS y Amazon SES para la entrega de alertas transaccionales y códigos de seguridad.



[Contexto del sistema] Sistema BP Banca en Línea

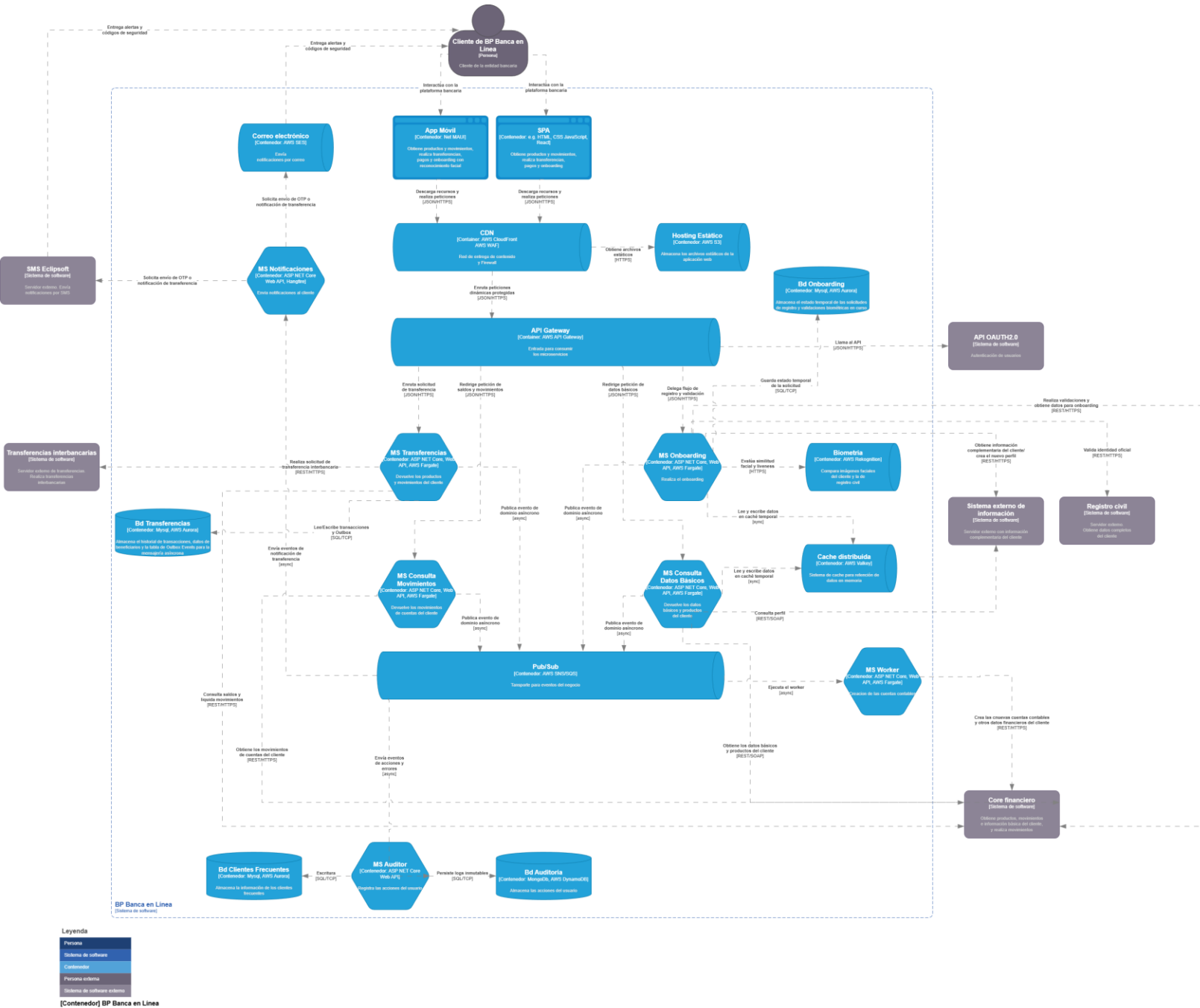
5.2. Nivel 2: Diagrama de Contenedores

Descripción de la Infraestructura y Dominios: Se visualizan los contenedores desplegados en AWS bajo una arquitectura de microservicios desacoplada y escalable:

- **Capa de Presentación y Borde:** Aplicaciones React y .NET MAUI que acceden a través de Amazon CloudFront y un API Gateway encargado del ruteo y la seguridad perimetral.
- **Capa de Microservicios:** Los dominios de negocio se ejecutan en contenedores independientes.

- **Capa de Persistencia y Mensajería:**

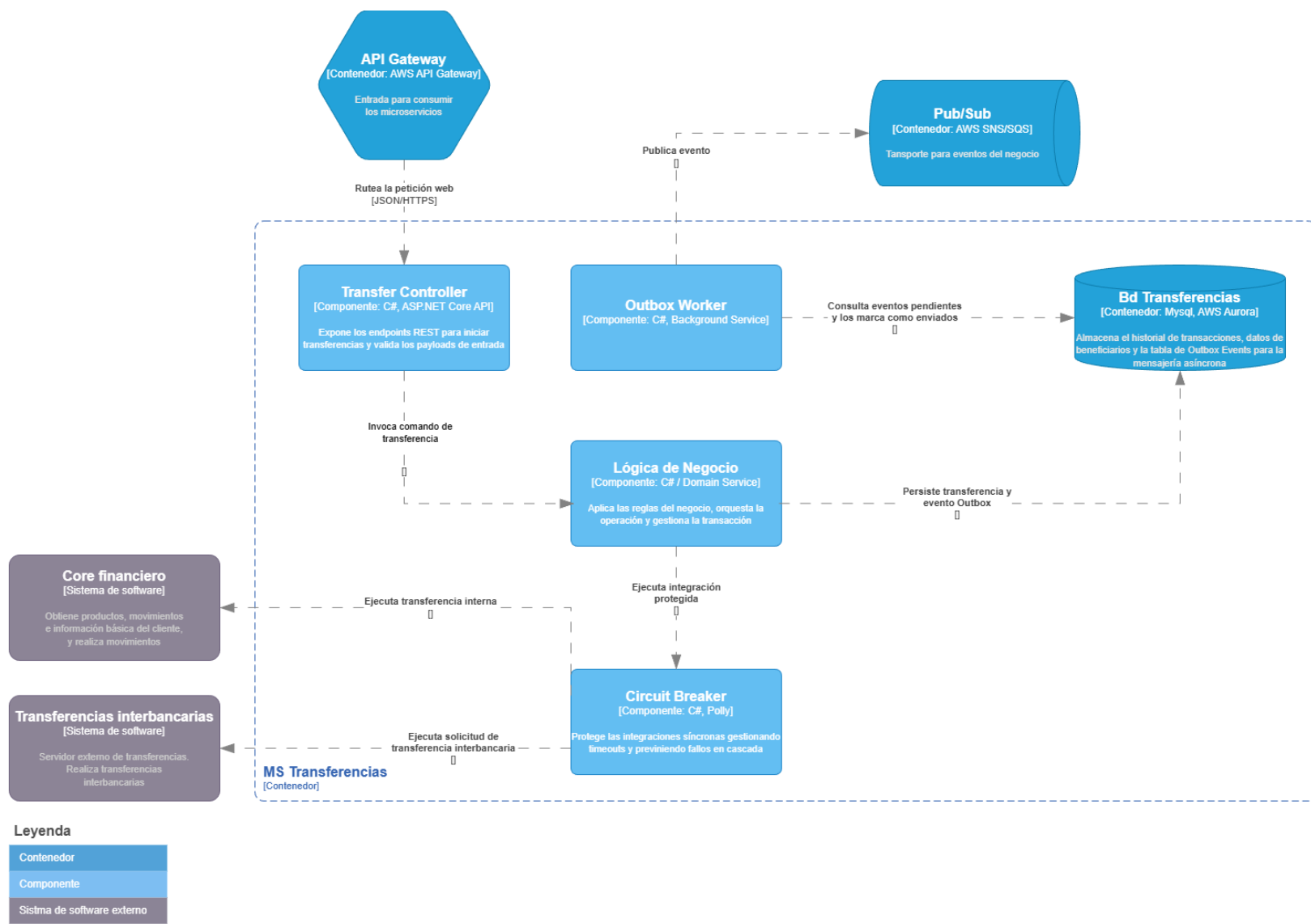
- **Amazon ElastiCache:** Almacén en memoria para optimizar la latencia de consultas recurrentes.
- **Amazon Aurora:** Motor relacional que garantiza la integridad ACID para transferencias y el modelo de lectura para cuentas frecuentes.
- **Amazon DynamoDB:** Almacén de alta ingesta para el registro inmutable de logs y auditoría.
- **Amazon SNS / SQS:** Bus de eventos encargado de la coreografía asíncrona entre servicios.



5.3. Nivel 3: Diagrama de Componentes y Patrones de Diseño

En este nivel se detallan los componentes internos y los patrones lógicos que permiten al microservicio de Transferencias resolver los retos de consistencia y resiliencia:

- **Patrón Transactional Outbox:** Es el corazón del microservicio. Para evitar la pérdida de mensajes, la Lógica de Negocio persiste en una única transacción atómica tanto el registro de la transferencia como el evento de notificación en la tabla de Outbox. Posteriormente, el Outbox Worker se encarga de leer estos eventos y asegurar su publicación en el bus de mensajes.
- **Patrón Circuit Breaker:** Este componente actúa como un escudo protector interceptando las llamadas hacia el Core Financiero y la Red Interbancaria. Si estos sistemas externos presentan lentitud o fallos críticos, el circuito se abre para evitar el agotamiento de recursos en el microservicio y permitir una respuesta de error controlada hacia el cliente.
- **Segregación de Responsabilidades:** El microservicio se libera de la carga de gestionar consultas complejas o históricos de auditoría. Su función es puramente transaccional; al finalizar cada operación, emite un evento de dominio al bus, delegando la actualización de las vistas de lectura y el registro de auditoría a otros componentes del sistema.
- **Controlador de API:** El Transfer Controller actúa como el punto de entrada, encargado de recibir las peticiones del API Gateway, validar la estructura de los datos y coordinar la ejecución de los comandos de negocio.



[Componente] BP Banca en Línea - MS Transferencias

6. Infraestructura Cloud, Alta Disponibilidad y Recuperación ante Desastres

Para cumplir con las exigencias normativas y garantizar la continuidad del negocio de la Banca en Línea BP, la arquitectura física se apoya en los pilares del AWS Well-Architected Framework.

6.1. Alta Disponibilidad (HA) y Tolerancia a Fallos

Despliegue Multi-AZ: Todos los componentes críticos se despliegan distribuidos en al menos tres Zonas de Disponibilidad distintas dentro de la misma región. Si un centro de

datos físico de AWS falla, el tráfico se enruta automáticamente a las zonas sanas sin impacto en el usuario.

Tolerancia a Fallos en Base de Datos: Amazon Aurora mantiene 6 copias de los datos distribuidas en 3 AZs. Si la instancia principal de escritura falla, el sistema promueve una réplica de lectura a principal en menos de 30 segundos de forma automática.

6.2. Auto-Healing y Excelencia Operativa

Health Checks y Auto-Scaling: Los microservicios en AWS Fargate están configurados con métricas de salud. Si un contenedor se bloquea, el orquestador lo destruye y levanta uno nuevo automáticamente. Además, mediante políticas de Auto-Scaling, si la CPU supera el 70%, se aprovisionan nuevos contenedores dinámicamente y se destruyen cuando la carga baja, optimizando el manejo de costos.

6.3. Recuperación ante Desastres y Backups

Estrategia Pilot Light / Warm Standby: Para cumplir con el Plan de Continuidad de Negocio, se establece una estrategia de DR en una región secundaria de AWS.

RPO y RTO: Se garantizan copias de seguridad continuas. Amazon Aurora permite un Recovery Point Objective de milisegundos gracias a su almacenamiento subyacente replicado, y DynamoDB utiliza Point-in-Time Recovery para restaurar la tabla a cualquier segundo de los últimos 35 días ante un borrado accidental de logs de auditoría.

6.4. Estrategia de Manejo de Costos

Dado que se cuenta con presupuesto para infraestructura en AWS, es imperativo aplicar principios de optimización de costos para evitar gastos ocultos por capacidad ociosa:

Computación Serverless: Al utilizar AWS Fargate, la entidad financiera no paga por servidores encendidos 24/7. El cobro se realiza estrictamente por los milisegundos de CPU y RAM consumidos durante la ejecución de las transferencias o consultas.

Capacity Planning en Bases de Datos: Aurora SQL permite el auto-escalado de almacenamiento y capacidad de cómputo, adaptándose automáticamente a los picos transaccionales, y reduciendo su tamaño en horas de bajo tráfico.

7. Conclusiones y Valor Agregado de la Solución

La arquitectura propuesta para la plataforma de Banca en Línea ha sido diseñada bajo los más altos estándares de la industria financiera, garantizando un equilibrio óptimo entre seguridad, rendimiento y mantenibilidad. Se destacan los siguientes pilares:

- **Seguridad de Grado Bancario:** La implementación del flujo OAuth 2.0 con PKCE, sumada a un esquema de autenticación de doble factor y el aislamiento de datos en redes privadas de AWS, asegura que la plataforma sea resiliente ante ataques

de interceptación y suplantación, protegiendo la integridad de los activos digitales de los clientes.

- **Consistencia e Integridad de Datos:** Mediante el uso de Amazon Aurora y el patrón Transaccional Outbox, la solución garantiza que ninguna transacción financiera se pierda o quede en un estado inconsistente. La aplicación de CQRS permite, además, que el sistema escale en volumen de consultas sin comprometer la estabilidad ni el rendimiento del Core Financiero.
- **Alta Disponibilidad y Resiliencia:** El uso de servicios gestionados y elásticos como AWS Fargate y DynamoDB, junto con la implementación de Circuit Breakers, asegura que la plataforma pueda absorber picos de tráfico y recuperarse automáticamente ante fallos de proveedores externos, garantizando la continuidad del servicio 24/7.
- **Eficiencia Operativa:** La selección de un stack tecnológico unificado en .NET / C# para el backend y la aplicación móvil, junto con el uso de React para la web, permite una reutilización masiva de lógica de negocio y estándares de validación. Esto reduce drásticamente el TCO y simplifica el mantenimiento evolutivo a largo plazo.